Ders 17 Çalışma Özeti

Ders 17 Çalışma Özeti

Genel Konular

- Dersin tarihi 16 Kasım 2021 Salı. Ders Zoom üzerinden yapıldı.
  - Bu kayıt kabuk (shell) programlama uygulamasının ikinci ve son oturumudur. Ders 15'teki uygulamanın devamıdır ve aynı öğretim elemanı anlatıyor.
  - Asıl anlatım yaklaşık 00:13'te başlar. Öncesi katılım beklemesidir.
- Önceki haftanın tekrarı ve eksik kalan konu
  - Kullanıcı ve grup dosyaları, `chmod`, dosya tipleri, `echo`, değişkenler, aritmetik, `read`, joker karakterler, > ile >> farkı.
- `mv` ile dosya adını değiştirme ve `sort`
- Boru (pipe, `|`) ile komutları birleştirme
  - `ps aux | grep`, `set | grep`.
- `tr` ile karakter dönüştürme ve silme
- `awk` ile sütun bazlı filtreleme
- `bc` hesap makinesi
- Koşul yapıları: `if`, `then`, `else`, `fi` ve `test` koşulları
- Döngüler: `for`, `while`, `until`
- `let` ile aritmetik ve `case` yapısı

Hocanın Özellikle Vurguladığı Kısımlar

- Boru (`|`) sol taraftaki komutun çıktısını sağ taraftaki komuta girdi olarak verir.
  - Uzun çıktılarda aradığınızı bulmak için `| grep` ile filtreleme yapın.
- Değişkenlere ve argümanlara `$` ile erişilir.
  - Örnek: `$1` betiğe verilen birinci argümandır.
- `case` yapısında her durum `;;` ile biter. Bu ayırıcıya dikkat edin.
- `case` ile yapılan her şey `if` ile de yapılabilir. `case` daha okunaklıdır.
- Betiği çalıştırırken argümanları vermeyi unutmayın.
  - Derste bir örnekte dosya adı argümanı verilmediği için ilk denemede beklenen sonuç alınmadı.
- Burada gösterilen temel yapılar herhangi bir kabuk betiği yazmak için yeterli.
  - Kabuk programlamada uzman olmanız gerekmiyor. Ama bir gün bir program kurarken bu komutlarla karşılaşacaksınız. Ne yaptıklarını anlamanız önemli.
- Proje bilgisi:
  - Uygulamalardan bir proje verilebilir. Problem verilecek, tasarım size kalacak.
  - Büyük ihtimalle bir çekirdek modülü (kernel module) istenecek, ama henüz kesin değil.
  - Sınavla ilgili sorularınızı dersin hocasına sorun.

Kısa Tekrar Notları

- Kullanıcı bilgileri `/etc/passwd`, şifre özetleri (hash) `/etc/shadow`, gruplar `/etc/group` dosyasındadır. Alanlar `:` ile ayrılır.
- `mv eski yeni`: dosya adını değiştirir. Aynı komut taşıma için de kullanılır.
- `sort`: satırları sıralar.
- ~: kullanıcının ev (home) klasörü.
- `ps aux`: çalışan bütün süreçleri listeler.

- `ps aux | grep -i root`: içinde “root” geçen satırları büyük-küçük harf ayırmadan gösterir.
- `set | grep USER`: `USER` geçen değişkenleri gösterir.
- `tr '[a-z]' '[A-Z]'`: küçük harfleri büyük harfe çevirir.
- `tr ';' ','`: noktalı virgülleri virgülle değiştirir.
- `tr -d ';'`: noktalı virgülleri siler.
- `awk -F ';' 'int($2)>3'`: `;` ile ayrılmış satırlarda ikinci sütunu 3’ten büyük olanları yazar.
  - `$1`, `$2`, `$3` sırayla birinci, ikinci, üçüncü sütundur.
- `bc`: hesap makinesi. Karşılaştırmalarda doğruysa 1, yanlışsa 0 döndürür.
- `if [ koşul ]; then ... else ... fi`
- Koşul işleçleri:
  - sayılar için `-gt` (büyük), `-lt` (küçük), `-eq` (eşit)
  - dosyalar için `-f` (dosya), `-d` (klasör), `-r` (okunabilir), `-w` (yazılabilir)
- `$#`: betiğe verilen argüman sayısı.
- `for` biçimleri:
  - `for i in 1 2 3 4 5`
  - `for i in {1..5}`
  - `for ((i=1; i<=5; i++))`
  - `for i in {1..10..2}`
  - `for i in $(ls)`
  - `for i in ~/*`
- Döngü gövdesi `do` ile başlar, `done` ile biter.
- `while [ $i -gt 0 ]` koşul doğru oldukça döner. `until` koşul doğru olana kadar döner.
- `let "sayac-=1"`: `sayac=$((sayac-1))` ile aynı işi yapar.
- `case $x in desen) komutlar ;; *) varsayılan ;; esac`

Detaylı Açıklamalar

Kullanıcıların listesi. Önceki hafta kullanıcı listesinin yeri hatırlanamamıştı. Bu derste `/etc/shadow` dosyası gösterildi. Önceki hafta oluşturulan test kullanıcıları burada görünüyordu. Satırlar `:` ile ayrılmış alanlardan oluşur. Birinci alan kullanıcı adı, ikinci alan şifrenin özetidir.

Boru ve filtreleme. `ps aux` sistemdeki bütün süreçleri gösterir; görev yöneticisinin terminaldeki karşılığıdır. Liste çok uzundur. Örneğin yalnızca root kullanıcısının süreçlerini görmek için:

```
ps aux | grep -i root
set | grep USER
```

`grep -i` büyük-küçük harf ayrımını kaldırır.

`tr` örneği. Slayttaki örnek sırayla şunları yapar:

```
echo "merhaba;dunya" | tr '[a-z]' '[A-Z]'   # MERHABA;DUNYA
echo "a;b;c" > dosya.txt
cat dosya.txt | tr ';' ','                  # a,b,c
cat dosya.txt | tr -d ';'                   # abc
```

`awk` örneği. Slayttaki dosya ve komutlar:

```
echo "ahmet;4;1" >  birdosya.txt
echo "ahmet;3;2" >> birdosya.txt
echo "ahmet;2;3" >> birdosya.txt
echo "ahmet;1;4" >> birdosya.txt
echo "ahmet;0;5" >> birdosya.txt
cat birdosya.txt | awk -F ';' 'int($2)>3'    # ahmet;4;1
cat birdosya.txt | awk -F ';' 'int($3)<=4'   # ilk dört satır
```

`-F ';'` alan ayırıcısını belirler. Ayırıcı yanlış verilirse (örneğin `,`) satır bölünemez: `$2` ve `$3` boş kalır, `int()` bunları 0 sayar. Bu yüzden `int($3)<=4` bütün satırları yazar, `int($2)>3` hiçbir satır yazmaz. CSV gibi tablo biçimli dosyalarda bu yöntem çok işe yarar.

Koşul örneği. Slayttaki sorgu betiği betiğe verilen argümanları kontrol eder: - Birinci argüman bir dosya adıdır. `if cat $1` ile dosya okunabiliyorsa “mevcut” yazılır. - `$#` ile argüman

sayısı 3'ten büyük mü diye bakılır. - Üçüncü ve dördüncü argümanların eşit olup olmadığı karşılaştırılır. - Son olarak `-r` ile dosyanın okunabilir, `-w` ile yazılabilir olduğu kontrol edilir.

Örnek çalıştırma: `./13_queries.sh dosya.txt 4 3 2 1`

Döngü örnekleri.

```
for i in {1..10..2}; do echo "sayac $i"; done      # 1 3 5 7 9

for i in ~/*; do
  if [ -f $i ]; then echo "dosya : $i"
  elif [ -d $i ]; then echo "klasor : $i"
  else echo "bilemedim bu ne : $i"
  fi
done

i=3
while [ $i -gt 0 ]; do
  echo "sayac $i"
  i=`expr $i - 1`
done                                               # 3 2 1
```

Kabukta kod blokları süslü parantezle ya da girintiyle değil, anahtar kelimelerle belirlenir: `if ... fi`, `do ... done`, `case ... esac`.

Sonraki uygulamalar. Kalan uygulamalar sırayla süreçler arası iletişim, iş parçacıkları ve çekirdek modülü olacak. Shell programlamayı da kapsayan bir proje verilebilir. Bu uygulamalar projeye başlangıç noktası olacak.

Ders sonrası notlar

- Derste `/etc/shadow` için "kullanıcıların bulunduğu dosya" dendi ve grupların da aynı dosyada olduğu söylendi. Doğrusu:
  - Kullanıcı listesi `/etc/passwd` dosyasındadır.
  - `/etc/shadow` şifre özetlerini tutar ve yalnızca root okuyabilir.
  - Gruplar `/etc/group` dosyasındadır.
- Derste `grep`'in "yer değiştirme (replacement) yapan bir fonksiyon" olduğu da söylendi. Bu yanlış. `grep` yalnızca arar ve eşleşen satırları yazar. Yer değiştirme için `sed` kullanılır.
- `if cat $1` koşulu "cat'in dosyanın var olup olmadığını bildiren parametresi" değildir. Kabukta her komut bir çıkış durumu (exit status) döndürür. `cat` başarılıysa 0 döner ve `if` bunu doğru kabul eder. `$1` ise betiğe verilen birinci argümandır.
  - Dosyanın var olup olmadığını kontrol etmenin doğru yolu `if [ -e "$1" ]` ya da `if [ -f "$1" ]` yazmaktır.
- Altyazıda yazılabilirlik için "çizgi V" geçiyor. Doğrusu `-w`'dir.
- Sorgu betiğinin derste çalıştırılmasında ilk denemede yalnız `3 4 5`, ikinci denemede `4 3 2 1` verildi; dosya adı verilmedi. Bu yüzden `$1` bir sayı oldu ve `cat` hata verdi ("3 isimli dosya mevcut değil"). Bu çıktılar bash ile yeniden üretildi. Doğru çalıştırma, dosya adını birinci argüman olarak vermektir.
- Awk slaytının açıklama satırında "tr, küçük harfleri büyük harfe çevirecektir" yazıyor. Bu satır önceki slayttan kopyalanmış, awk örneğiyle ilgisi yok.
- Awk slaytında beş `echo` komutu tek satırda ve aralarında `;` olmadan yazılmış. Bu hâliyle çalışmaz. Komutlar ayrı satırlara yazılmalı ya da `;` ile ayrılmalıdır.
- `tr`, `awk`, `for` ve `while` örneklerinin çıktıları bash'te çalıştırılarak doğrulandı.
- Bu uygulamada kabuk fonksiyonları (`17_function.sh`) gösterilmedi. Slaytlarda da yok.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.